src/rag.py

Naive single-shot RAG — the baseline the agent must beat

This module implements **classic RAG**: retrieve the top chunks once, put them in a grounded prompt, and generate one answer. It is deliberately the simple baseline; its built-in weakness on multi-hop questions is what motivates the agent in agent.py.

Walkthrough of all 90 lines, in order. Left numbers are the real line numbers in the file.

```
 1  """Naive (single-shot) RAG — the baseline the agent has to beat.
 2
 3  The classic RAG pipeline, in three moves:
 4
 5      retrieve once  ->  stuff the chunks into a grounded prompt  ->  generate
 6
 7  It retrieves the top-k chunks for the question, hands them to Claude with strict
 8  instructions to answer *only* from those chunks and cite them, and returns the
 9  answer plus the sources. There is no iteration: one retrieval, one generation.
10
11  This is the honest baseline. Its weakness is built in — if the single retrieval
12  misses a fact (common on multi-hop questions that need two different pieces of
13  evidence), the model never gets a second chance. The agentic version in agent.py
14  exists precisely to fix that, and the evaluation measures whether it does.
15  """
```

Lines 1 to 15 are the module docstring. They summarise RAG as three moves — retrieve once, stuff the chunks into a grounded prompt, generate — and name the built-in weakness: a single retrieval that misses a fact gives the model no second chance, which is common on multi-hop questions and is exactly what the agent fixes.

```
16
17  from __future__ import annotations
18
19  from dataclasses import dataclass, field
20
21  from . import config, llm
22  from .vectorstore import Retrieved, VectorStore
```

Lines 16 to 22 are imports. Line 17 enables postponed annotation evaluation. Line 19 imports dataclass and field for the small result type defined below. Line 21 imports the project config and the llm wrapper. Line 22 imports Retrieved (one search hit) and VectorStore (the shared retriever) from the vector-store module.

```
23
24  # The grounding contract. Two rules do the real work: answer ONLY from the
25  # provided context (this is what stops the model from falling back on its
26  # parametric memory and hallucinating), and cite the arXiv ids you used (this is
27  # what makes the answer auditable). Telling it to say when the context is
28  # insufficient is what lets "I don't know" be a correct answer instead of a
29  # confident wrong one.
30  SYSTEM_PROMPT = (
31      "You are a research assistant answering questions about machine-learning "
32      "papers. Answer using ONLY the provided context passages. Every claim must be "
33      "grounded in the passages — do not use outside knowledge. Cite the arXiv ids "
34      "of the passages you used, in square brackets, e.g. [2401.01234]. If the "
35      "passages do not contain enough information to answer, say so plainly rather "
36      "than guessing."
37  )
```

Lines 23 to 37 define SYSTEM_PROMPT. The comment on lines 24 to 29 explains the grounding contract; the string on lines 30 to 37 is the actual instruction sent to Claude. Its three working rules: answer ONLY from the provided passages (blocks the model from using memory and hallucinating), cite the arXiv ids used in square brackets (makes the answer auditable), and say plainly when the passages are insufficient (makes I-don't-know a valid answer).

```
38
39
40  @dataclass
41  class RAGAnswer:
42      """A grounded answer plus everything needed to evaluate and audit it."""
43
44      question: str
45      answer: str
46      sources: list[Retrieved]
47      cited_ids: list[str] = field(default_factory=list)
```

Lines 38 to 47 define the RAGAnswer dataclass. The @dataclass decorator (line 40) auto-generates the constructor. The fields are the question asked (line 44), the generated answer text (line 45), the list of Retrieved source chunks (line 46), and cited_ids (line 47) — which defaults to an empty list via field(default_factory=list) so each instance gets its own list rather than sharing one.

```
48
49
50  def format_context(hits: list[Retrieved]) -> str:
51      """Render retrieved chunks into a numbered, id-tagged context block.
52
53      Each passage is labelled with its arXiv id so the model can cite it and a
54      reader can trace every claim back to a specific chunk.
55      """
56      return "\n\n".join(
57          f"[{h.arxiv_id}] {h.title}\n{h.text}" for h in hits
58      )
```

Lines 48 to 58 define format_context, which renders the retrieved chunks into the text block placed in the prompt. The docstring (lines 51 to 55) notes each passage is tagged with its arXiv id so the model can cite it. The return (lines 56 to 58) joins the hits with blank lines, each formatted as its id in brackets, its title, then its text — a generator expression iterating over the hits.

```
59
60
61  def extract_cited_ids(answer: str, hits: list[Retrieved]) -> list[str]:
62      """Find which retrieved arXiv ids the answer actually cited.
63
64      We only count ids that were in the retrieved set — a "citation" of something
65      that wasn't retrieved is a hallucinated reference, not a real source.
66      """
67      retrieved_ids = {h.arxiv_id for h in hits}
68      return [aid for aid in retrieved_ids if f"[{aid}]" in answer]
```

Lines 59 to 68 define extract_cited_ids, the citation-honesty check. Line 67 builds a set of the ids that were actually retrieved. Line 68 returns only those retrieved ids whose bracketed form appears in the answer text. The docstring (lines 62 to 66) explains why: a citation to something never retrieved is a hallucinated reference and must not be counted as a real source.

```
69
70
71  def answer(store: VectorStore, question: str, k: int = config.TOP_K) -> RAGAnswer:
72      """Run naive RAG end to end and return a grounded, cited answer."""
73      hits = store.search(question, k=k)
74      context = format_context(hits)
75      user = (
76          f"Context passages:\n\n{context}\n\n"
77          f"Question: {question}\n\n"
78          "Answer using only the passages above, and cite the arXiv ids you used."
79      )
80      result = llm.call(
81          messages=[{"role": "user", "content": user}],
82          system=SYSTEM_PROMPT,
83      )
84      text = llm.first_text(result)
85      return RAGAnswer(
86          question=question,
87          answer=text,
88          sources=hits,
89          cited_ids=extract_cited_ids(text, hits),
90      )
```

Lines 69 to 90 define answer(), the end-to-end naive pipeline. It takes the shared store, the question, and k (defaulting to config.TOP_K). Line 73 retrieves the top-k chunks. Line 74 formats them into context. Lines 75 to 79 build the user message: the context, then the question, then a restatement of the instruction. Lines 80 to 83 call the LLM with that message and the system prompt. Line 84 pulls the plain text out of the response. Lines 85 to 90 package everything into a RAGAnswer, computing cited_ids from the final text so the record is self-contained and auditable.